

Scalable AI

Breaking lecture: Llama 3!

Meta svela Llama 3, la sua IA più potente. Ma in Italia la useremo chissà quando

di Pier Luigi Pisa

Il nuovo modello di IA open-source sarà integrato nelle principali piattaforme Meta, da Instagram a disposizione degli sviluppatori. Ma in Italia la Meta AI non è ancora disponibile e non ci sono novità



Build the future of AI with Meta Llama 3

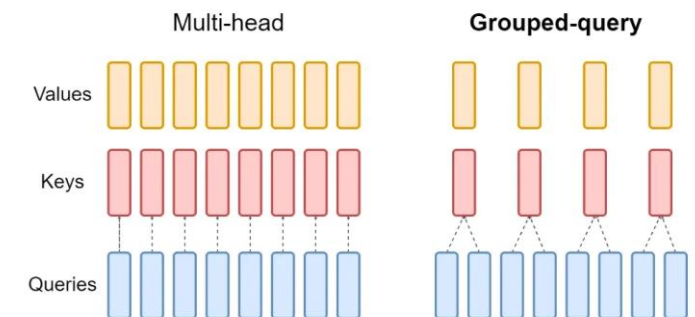
Now available with both 8B and 70B pretrained and instruction-tuned versions to support a wide range of applications

Meta lancia Llama 3 e prepara già un'AI ancora più potente

Dopo aver presentato il modello di intelligenza artificiale open source più capace in circolazione, il colosso ha fatto sapere di aver in cantiere un erede addirittura superiore

Technical Features (from @karpathy)

- Released in 8B and 70B variants, both in their pre-trained and instruction fine-tuned versions
- The 400B version is still training 😊
- **Tokenizer:** number of tokens in the dictionary was 4X'd from 32K (Llama 2) -> 128K (Llama 3). With more tokens you can compress sequences more in length, cites 15% fewer tokens, and see better downstream performance.
- **Architecture:** no major changes from the Llama 2. In Llama 2 only the bigger models used Grouped Query Attention (GQA), but now all models do, including the smallest 8B model. This is a parameter sharing scheme for the keys/values in the Attention. In practice, instead of having all key/values heads talking to all query heads, each key/value head only talks to a subset (“group”) of the query heads.



Technical Features (from @karpathy)

- **Sequence length:** the maximum number of tokens in the context window was bumped up to 8192 from 4096 (Llama 2) and 2048 (Llama 1). This bump is welcome, but quite small w.r.t. modern standards (e.g. GPT-4 is 128K).
- **Training data.** Llama 2 was trained on 2 trillion tokens, Llama 3 was bumped to 15T training dataset, including a lot of attention that went to quality, 4X more code tokens, and 5% non-en tokens over 30 languages. (5% is fairly low w.r.t. non-en:en mix, so certainly this is a mostly English model, but it's quite nice that it is > 0).
- **Scaling laws.** Very notably, 15T is a very very large dataset to train with for a model as "small" as 8B parameters, and this is not normally done and is new and very welcome. The "compute optimal" point for an 8B model, according to the scaling laws, would be train it for $\sim 200B$ tokens. So this is training $\sim 75X$ beyond that point, which is unusual. Meta mentions that even at this point, the model doesn't seem to be "converging" in a standard sense. In other words, the LLMs we work with all the time are significantly undertrained by a factor of maybe 100-1000X or more, nowhere near their point of convergence.
- **Systems.** Llama 3 is cited as trained with 16K GPUs at observed throughput of 400 TFLOPS.

About Instruction Tuning

“To fully unlock the potential of our pretrained models in chat use cases, we innovated on our approach to instruction-tuning as well. Our approach to post-training is a **combination of supervised fine-tuning (SFT), rejection sampling, proximal policy optimization (PPO), and direct policy optimization (DPO)**. The quality of the prompts that are used in SFT and the preference rankings that are used in PPO and DPO has an outsized influence on the performance of aligned models. **Some of our biggest improvements in model quality came from carefully curating this data and performing multiple rounds of quality assurance on annotations provided by human annotators.**

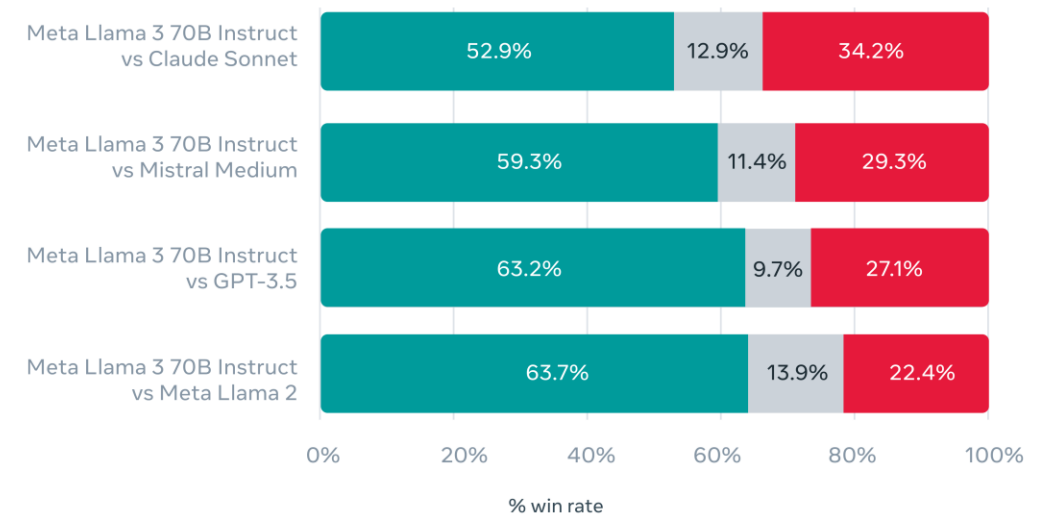
Learning from preference rankings via PPO and DPO also greatly improved the performance of Llama 3 on reasoning and coding tasks. We found that if you ask a model a reasoning question that it struggles to answer, the model will sometimes produce the right reasoning trace: The model knows how to produce the right answer, but it does not know how to select it. Training on preference rankings enables the model to learn how to select it.”

Performance (from Meta blog post)

- With Llama 3, we set out to build the best open models that are on par with the best proprietary models available today.
- Our new 8B and 70B parameter Llama 3 models establish a new state-of-the-art for LLM models at those scales. Thanks to improvements in pretraining and post-training, our pretrained and instruction-fine-tuned models are the best models existing today at the 8B and 70B parameter scale.
- We developed a new high-quality human evaluation set. This evaluation set contains 1,800 prompts that cover 12 key use cases: asking for advice, brainstorming, classification, closed question answering, coding, creative writing, extraction, inhabiting a character/persona, open question answering, reasoning, rewriting, and summarization. The chart below shows aggregated results of our human evaluations across of these categories and prompts against Claude Sonnet, Mistral Medium, and GPT-3.5.

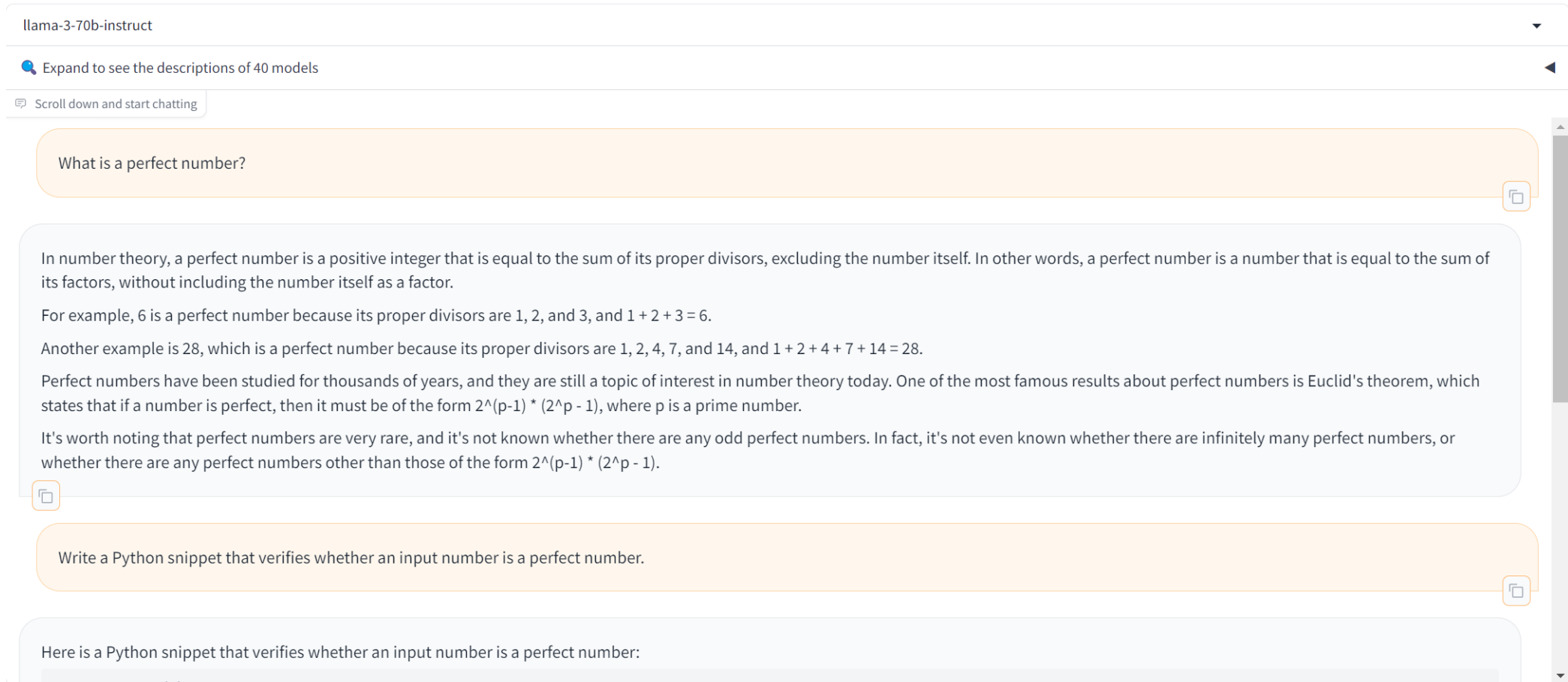
Meta Llama 3 Instruct model performance

	Meta Llama 3 8B	Gemma 7B - It Measured	Mistral 7B Instruct Measured		Meta Llama 3 70B	Gemini Pro 1.5 Published	Claude 3 Sonnet Published
MMLU 5-shot	68.4	53.3	58.4	MMLU 5-shot	82.0	81.9	79.0
GPQA 0-shot	34.2	21.4	26.3	GPQA 0-shot	39.5	41.5 CoT	38.5 CoT
HumanEval 0-shot	62.2	30.5	36.6	HumanEval 0-shot	81.7	71.9	73.0
GSM-8K 8-shot, CoT	79.6	30.6	39.9	GSM-8K 8-shot, CoT	93.0	91.7 11-shot	92.3 0-shot
MATH 4-shot, CoT	30.0	12.2	11.0	MATH 4-shot, CoT	50.4	58.5 Minerva prompt	40.5



Let's try it (1)

- Go to <https://chat.lmsys.org/>, select «Direct chat» and llama3-70b-instruct.



The screenshot shows a web interface for the llama3-70b-instruct model. At the top, there's a header with the model name and a search icon. Below the header, there's a search bar with the text "Expand to see the descriptions of 40 models". A button labeled "Scroll down and start chatting" is visible. The main chat area contains two messages. The first message is a user query: "What is a perfect number?". The second message is the model's response, which explains the definition of a perfect number, provides examples (6 and 28), and discusses Euclid's theorem. The response is formatted with paragraphs and a small icon at the bottom left. The third message is another user query: "Write a Python snippet that verifies whether an input number is a perfect number.". The fourth message is the start of the model's response: "Here is a Python snippet that verifies whether an input number is a perfect number:". The interface includes a scrollbar on the right side.

llama-3-70b-instruct

Expand to see the descriptions of 40 models

Scroll down and start chatting

What is a perfect number?

In number theory, a perfect number is a positive integer that is equal to the sum of its proper divisors, excluding the number itself. In other words, a perfect number is a number that is equal to the sum of its factors, without including the number itself as a factor.

For example, 6 is a perfect number because its proper divisors are 1, 2, and 3, and $1 + 2 + 3 = 6$.

Another example is 28, which is a perfect number because its proper divisors are 1, 2, 4, 7, and 14, and $1 + 2 + 4 + 7 + 14 = 28$.

Perfect numbers have been studied for thousands of years, and they are still a topic of interest in number theory today. One of the most famous results about perfect numbers is Euclid's theorem, which states that if a number is perfect, then it must be of the form $2^{p-1} \cdot (2^p - 1)$, where p is a prime number.

It's worth noting that perfect numbers are very rare, and it's not known whether there are any odd perfect numbers. In fact, it's not even known whether there are infinitely many perfect numbers, or whether there are any perfect numbers other than those of the form $2^{p-1} \cdot (2^p - 1)$.

Write a Python snippet that verifies whether an input number is a perfect number.

Here is a Python snippet that verifies whether an input number is a perfect number:

Let's try it (1)

- Go to <https://chat.lmsys.org/>, select «Direct chat» and llama3-70b-instruct.

llama-3-70b-instruct

Expand to see the descriptions of 40 models

Scroll down and start chatting

Here is a Python snippet that verifies whether an input number is a perfect number:

```
def is_perfect(n):  
    sum = 0  
    for i in range(1, n):  
        if n % i == 0:  
            sum += i  
    return sum == n  
  
# Test the function  
num = int(input("Enter a number: "))  
if is_perfect(num):  
    print(f"{num} is a perfect number!")  
else:  
    print(f"{num} is not a perfect number.")
```

Here's how the code works:

1. The `is_perfect` function takes an integer `n` as input.
2. It initializes a variable `sum` to 0, which will store the sum of the proper divisors of `n`.
3. It loops through all numbers `i` from 1 to `n-1` (since `n` itself is not a proper divisor).
4. For each `i`, it checks if `n` is divisible by `i` using the modulo operator (`n % i == 0`). If it is, it adds `i` to the `sum`.
5. Finally, it returns `True` if the `sum` is equal to `n`, and `False` otherwise.
6. The main code prompts the user to enter a number, and then calls the `is_perfect` function with that number as input. If the function returns `True`, it prints a message indicating that the number is perfect; otherwise, it prints a message indicating that it's not.

Note that this implementation has a time complexity of $O(n)$, which can be slow for large inputs. There are more efficient algorithms for testing perfect numbers, but this simple implementation is sufficient for small to moderate-sized inputs.

Let's try it (1)

- Go to <https://chat.lmsys.org/>, select «Direct chat» and llama3-70b-instruct.
- "x - 10 + 14 = 0 what is x? "
- "Who is the current prime minister of Italy?"
- "Give me the recipe for tortellini"
- "A brother and a sister were born in summer and in winter. The sister was not born in winter. Who was born in summer? "
- "I sat on the beach under an umbrella at midnight for ten hours, am I sunburned?"